

Revisiting Stream-Cipher-Based Homomorphic Transciphering in the TFHE Era

Adda-Akram Bendoukha, Aymen Boudguiga(✉), and Renaud Sirdey

Université Paris-Saclay, CEA-List, Palaiseau, France
{adda-Akram.bendoukha,aymen.boudguiga,renaud.sirdey}@cea.fr

Abstract. Transciphering allows to workaround the large expansion of the size of FHE encrypted data, thanks to the use of symmetric cryptography. Transciphering is a recryption technique that delegates the effective homomorphic encryption to the cloud. As a result, a client only has to encrypt (once) a symmetric key SYM.sk under a homomorphic encryption system, while his payload data are encrypted under SYM.sk using the chosen symmetric encryption algorithm.

In this work, we study the performances of some symmetric encryption algorithms in light of the TFHE cryptosystem and its properties. This allows us to unleash the use of additional existing symmetric algorithms which were not viable candidates for efficient encrypted domain execution with levelled-FHEs. In particular, we provide experimental evidences that Grain128-AEAD, a well established and well respected stream-cipher which is a finalist of the NIST competition for light-weight cryptography, is amenable to practical performances when run in the encrypted domain. As such, our work extends practical transciphering capabilities to include authenticated encryption for the first time.

Keywords: FHE · Stream-ciphers · Transciphering

1 Introduction

Fully Homomorphic Encryption (FHE) provides end-to-end confidentiality of personal and sensitive data by allowing general computations in the encrypted domain. Indeed, FHE is well suited for services requiring clients data offloading to computation servers in the Cloud. Yet, such offloading necessarily implies sending data over a network at some point, and the intrinsic expansion factor of FHE ciphertexts becomes an issue for their transmission and a potential show stopper for FHE usage with some use-cases. Indeed, large FHE ciphertexts are a

The research leading to these results has been funded in part from the European Union's Preparatory Action on Defence Research (PADR-FDDT-OPEN-03-2019). This paper reflects only the authors' views and the Commission is not liable for any use that may be made of the information contained therein.

© Springer Nature Switzerland AG 2022
E. Aïmeur et al. (Eds.): FPS 2021, LNCS 13291, pp. 19–33, 2022.
https://doi.org/10.1007/978-3-031-08147-7_2

bottleneck, especially in infrastructures with small bandwidth such as IoT infrastructures using for example LoRa or Sigfox for communication with a gateway to the cloud.

Transciphering works around this expansion issue by combining both symmetric cryptography and FHE. This results in a mean for compressing FHE ciphertexts prior to their transmission, with the ability to recover the homomorphic encryption of the message later without any security breaches and with additional computational cost only on the server.

While transciphering has been studied in depth in the state-of-the-art between 2015 and 2018 [3,4,8,9,19,21–23], most existing works have done so in the context of levelled FHE cryptosystems which suffer from limitations due to the increased computational cost of FHE calculations at larger multiplicative depth. As such, there is a need to revisit these works as well as the portfolio of symmetric algorithms amenable to practical performances when run in the encrypted domain in light of the TFHE cryptosystem which enabled practical bootstrapping and which, as such, does not induce multiplicative-depth related limitations. To the best of our knowledge, this has not yet been done in the state of the art.

Contribution – In this work we investigate stream-cipher-based transciphering by means of TFHE. We consider the following steam-ciphers: Trivium [15], Kreyvium [8], Grain-128a [1] and Grain128-AEAD [18]. We show that TFHE leads to practical performances when transciphering with the aforementioned stream-ciphers. In particular, as it removes the burden of multiplicative-depth constraints imposed by levelled FHE schemes, we show that TFHE allows to perform transciphering with a much smaller amount of homomorphic calculations (requiring to perform only one rather than many warm-ups in the encrypted domain). Also, this paper is, to the best our of knowledge, the first to demonstrate practical performances when running Grain-128a and Grain-AEAD in the FHE domain and to bring anthenticated encryption into the picture of transciphering (thanks to Grain-AEAD). In addition, we discuss several optimized implementations for these stream-ciphers with words which are 8 or 32-bits long. Finally, we detail the timing performances obtained when running each of these algorithms over TFHE by means of Cingulata [12], a well-established homomorphic applications compiler.

Paper Organization – The remainder of this paper is organized as follows. In Sect. 2, we briefly recall the principle of transciphering. In Sect. 3, we briefly present TFHE as our target FHE scheme for transciphering. In Sect. 5 and 6, we review the stream-ciphers that we intend to use for transciphering with TFHE. And in the last Sect. 7, we discuss implementation details and performance results.

2 Tranciphering

In a typical homomorphic encryption usage scenario, a first party (e.g., a client) aims at delegating the evaluation of a certain function f over a confidential message m to a distant Cloud server. In a direct approach, The client would encrypt m using a Fully Homomorphic Encryption scheme (FHE). The size expansion

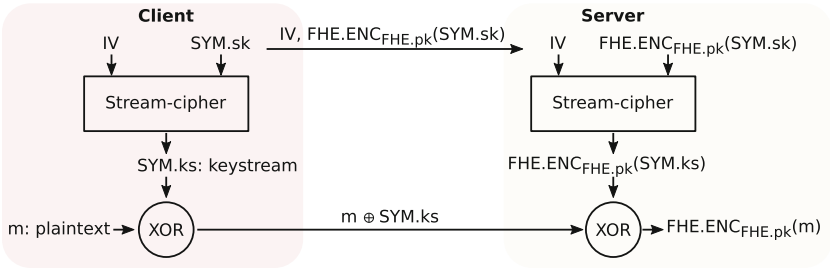


Fig. 1. Transciphering with a stream-cipher

factor is defined as $\frac{|\mathsf{FHE.Enc}(m)|}{|m|}$. In most FHE cryptosystems this factor is very large. In can reach some mega-bytes with respect to the chosen cryptosystem, its parameters and the chosen security level. This by opposition to protocols involving symmetric algorithms which usually lead to negligible or small overheads. This fact yielded the idea to bind FHE and symmetric cryptography to workaround this size expansion issue.

Indeed, with transciphering, we aim at lessening the quantity of homomorphically encrypted data to be sent to the Cloud by encrypting it symmetrically instead, and preserving the ability to recover the entire homomorphic encryption of the initial data. To do so, a client encrypts his message m using a symmetric cryptosystem with key $\mathsf{SYM.sk}$, and the key $\mathsf{SYM.sk}$ is encrypted using a homomorphic cryptosystem with key $\mathsf{FHE.pk}$. At the reception of $(\mathsf{SYM.Enc}_{\mathsf{SYM.sk}}(m), \mathsf{FHE.Enc}_{\mathsf{FHE.pk}}(\mathsf{SYM.sk}))$, the Cloud server homomorphically runs the symmetric cryptosystem decryption function i.e., runs $\mathsf{SYM.Enc}$ using $\mathsf{FHE.Enc}_{\mathsf{FHE.pk}}(\mathsf{SYM.sk})$. In the case of stream-cipher, an FHE encrypted keystream is the output of the previous evaluation. On the Cloud side, we would have both $m \oplus keystream$ and $\mathsf{FHE.Enc}$(keystream). $\mathsf{FHE.Enc}$(m) is then easily computed by performing a homomorphic xor between $m \oplus keystream$ and $\mathsf{FHE.Enc}$(keystream) (as depicted in Fig. 1). The size of m can be arbitrarily large, while the size of $\mathsf{SYM.sk}$ is fixed and is usually small enough to be homomorphically encrypted and transmitted once and for all. This is how compression is obtained (at the computational cost, of course, of homomorphically running the symmetric scheme decryption function on the server).

In terms of security, as most practically-used symmetric encryption algorithms do not have formally established indistinguishability properties, it should be emphasized that using transciphering jeopardizes the IND-CPA property of the FHE scheme (FHE schemes can be at most IND-CCA1 and most the schemes used in practice are only IND-CPA). This should however not be considered an issue in practice provided that symmetric encryption more often than not teams with provably-secure public-key encryption for efficiency reasons in practical scenarios, and FHE is no exception. Nevertheless, if we assume a perfect PRF on the symmetric side, the resulting construction would be IND-CPA [10].

Yet, one important point is to choose the key size of the symmetric encryption algorithm consistently with the parameters of the FHE scheme. At present, common practice generally targets FHE security parameters λ of around 128 bits and not more as FHE performances significantly decreases in the parameter regimes of larger λ. As such, at present, transciphering should consider symmetric algorithms with 128 bits keys.

3 TFHE

In this section, we present a high-level description of TFHE to justify its suitability for transciphering.

3.1 Learning with Errors

The Learning With Errors (LWE) assumption states that given an arbitrary number of samples $(a, b = \langle a, s\rangle + e) \in \mathbb{Z}_q^{n+1}$ where a and s are both in $\mathbb{Z}_q^n$ and $e \in \mathbb{Z}_q$ is a small error sampled from a Gaussian distribution over $\mathbb{Z}_q$, finding s is a computationally hard problem. The hardness of LWE has been proved by Regev in [24].

LWE is adapted to the real torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ (i.e. $(a, b) \in \mathbb{T}^n \times \mathbb{T}$). We call TLWE and TRLWE the LWE instances over the torus and the polynomial ring $\mathbb{T}_N[X] = \mathbb{T}[X]/(X^N + 1)$, respectively. For a matter of efficiency binary keys are used in both TLWE and TRLWE

3.2 TLWE/TRLWE Encryption

Given a TLWE sample $(a, b = \langle a, s\rangle + e)$ and a message $m \in \mathbb{Z}_p$. Integer messages are mapped to elements of $\mathbb{T}$. The encryption of an integer m is given by $(a, b) + (0, m')$ where $m' = \frac{q}{p} \cdot \bar{m}$ and $\bar{m}$ the mapping of m over $\mathbb{T}$.

The decryption computes a dot product and then rounds the result to the nearest integer: $\frac{p}{q} \cdot (b, a)^T \cdot (1, -s) = \frac{p}{q} \cdot \lfloor \frac{q}{p} \cdot \bar{m} + e \rceil = \bar{m}$.

The same idea applies to the TRLWE settings. The aforementioned encryption scheme is an additive homomorphic scheme. In TFHE, the multiplication is ensured using GSW samples and a gadget decomposition algorithm [13,17].

3.3 Bootstrapping

In order to decrypt a message correctly, the noise e has to remain under a certain bound $e < \frac{q}{2 \cdot p}$. Obviously, the noise grows while evaluating operations on a ciphertext. The original idea of Gentry [16] was to boostrap a ciphertext to bring its noise to a fixed level. Bootstrapping consists in decrypting a ciphertext homomorphically. In practice, TFHE proposes the most efficient bootstrapping that takes 13 milliseconds per boolean gate computation. TFHE bootstrapping is explained in details in [13].

3.4 Impact on Transciphering

Thanks to its bootstrapping, TFHE offers greater freedom in the choice of the symmetric cryptosystem used for transciphering. Indeed with TFHE, we can perform more operations and even relax the constraints imposed by the multiplicative depth[1], since we have an efficient way to reduce the noise. This gives us access to a whole new set of symmetric cryptosystems eligible for transciphering.

In parallel, when stream-ciphers are used for tranciphering with a levelled homomorphic encryption scheme such as BFV, we have to bound their multiplicative depth. That is, we fix the size of the output keystream in advance with respect to the size of the data chunk to be encrypted. Bounding the stream-cipher size is suited for parallel implementation and batching. Otherwise, higher parameters have to be chosen for the cryptosystem in order to handle the required multiplicative depth. This solution has an impact on the general performance, not only of transciphering, but also on the computation on the message m initially planned.

4 Choice of the Symmetric Scheme

When it comes to transciphering, as stated in [8], stream-ciphers are more attractive than block-ciphers for two main reasons. First, when encrypting using a stream-cipher, keystream generation is independent of the data to be encrypted/decrypted. Consequently, almost all of the homomorphic calculations required for transciphering (i.e., the homomorphic keystream calculations) can be performed offline, outside of the critical latency path. Then, when the data to be tranciphered are available, there only remains to homomorphically XOR them with the precomputed homomorphically encrypted keystream. So, stream-cipher-based transciphering offers this advantage over block-cipher-based transciphering.

Second, modern stream-ciphers generally follow the pattern of a relatively expensive warm-up phase which enables the subsequent generation of an (almost) arbitrary long sequence of keystream. The keystream bits are generated in a very lightweight way. This is more interesting than block-cipher-based designs, since the warm-up has to be paid only once (or infrequently) and can be done offline. Note that one block-cipher run is more or less equivalent to one stream-cipher warm-up, in terms of computational cost.

Still, as shown in [8], when using levelled FHE scheme the warm-up had to be frequently redone (stream-ciphers were then preferred because they have an intrinsically lower multiplicative depth than block-ciphers) leading to a kind of CTR mode of operation for an IV-based stream-cipher such as Trivium. However, in this work, we remove the multiplicative-depth constraint by using TFHE, and avoid these regular warm-ups. As such, we enable a more natural use of a stream-ciphers with one (rather expensive) homomorphic warm-up for many (cheap)

[1] The multiplicative depth of a circuit is the maximum number of successive multiplications in the circuit.

homomorphic keystream bits generation (which can still be performed offline). As a consequence, the use of TFHE allows to drastically reduce the amount of homomorphic operations required for transciphering[2].

5 Trivium/Kreyvium

5.1 Trivium

When we search for a suitable stream-cipher to build an efficient transciphering scheme, we tend to give a particular interest to hardware-oriented and light-weight cryptosystems. Since they focus on providing a high throughput while using a small internal state and small arithmetic circuits, and of course an appropriate level of security. These specifications are quite similar to the ones we have for building transciphering schemes.

Trivium is part of the eSTREAM portofolio for hardware oriented encryption and fits the aforementioned requirements. Indeed, it is able to produce up to 2^{64} keystream bits from an 80-bits KEY and IV, by updating an internal state (S_i) composed of 3 registers of sizes 93, 84, and 111 by the mean of additions and multiplications and reinserting the output of each round back into the internal state (which is 288-bits long).

$$
\begin{aligned}
t_1 &= s_{66} + s_{93} + s_{91} \cdot s_{92} + s_{171} \\
t_2 &= s_{162} + s_{171} + s_{175} \cdot s_{176} + s_{264} \\
t_3 &= s_{243} + s_{288} + s_{286} \cdot s_{287} + s_{69} \\
(s_1, \ldots, s_{93}) &\longleftarrow (t_3, s_1, \ldots, s_{92}) \\
(s_{94}, \ldots, s_{177}) &\longleftarrow (t_1, s_{94}, \ldots, s_{176}) \\
(s_{178}, \ldots, s_{288}) &\longleftarrow (t_2, s_{178}, \ldots, s_{287})
\end{aligned}
$$

Trivium's structure can be viewed as a composition of three main functions. First, an initialization step (`Init(KEY, IV)`) sets the internal state value using bits from KEY, IV and zeros. Second, a `Warmup` step increases the internal state's entropy by iterating 1152 times, linear and non-linear operations, to mix bits from the different registers. Finally, a keystream generation step (`KeyGen(t)`), similar to `Warmup`, outputs the tth keystream bit by computing $z_t =$ `KeyGen(t)` $\circ$ `Warmup` $\circ$ `Init(KEY, IV)`. For more details about Trivium specification and security analysis, interested readers can refer to [15].

5.2 Kreyvium

Canteaut et al. [8] proposed Kreyvium to increase Trivium security from 80-bits to 128-bits. That is why, the design of Kreyvium is almost identical to Trivium. However, Kreyvium has an 128-bits key and IV each. The internal state of Kreyvium is composed of an extra pair of 128-bits registers initialized with the key and the IV. Thus, Kreyvium has 544-bits internal state. The key and IV bits are constantly re-injected by a XOR operation into the computation of the state update.

[2] As a general guidelines, the first rule of optimization for FHE is to find (legitimate) ways of doing less FHE calculations.

6 Grain128a/Grain128-AEAD

6.1 Grain128a

Grain128a [1] is a hardware-oriented stream-cipher from the eSTREAM portofolio and thus, a lightweight cryptosystem with well-respected security. It supports a 128-bits key and a 128-bits IV. Its internal state is composed of an LFSR and an NFSR[3], initialized respectively with the key and the IV. It comes with the possibility to perform an authenticated encryption by the mean of a MAC of size 32-bits.

6.2 Grain128-AEAD

Grain128-AEAD is a well-established stream-cipher which is widely inspired from Grain128a. It is a finalist of the NIST competition on lightweight cryptography and has slight modifications compared with its predecessor. Its Authenticated Encryption with Associated Data mode (AEAD) is compulsory and refers to the possibility of encrypting a subset of the plaintext bits using a mask d as: $c_i = m_i \oplus (ks_i \cdot d_i)$, as well as computing a larger MAC (64-bits long) on it. Grain128-AEAD internal state is 256-bits long and is formed by a 128-bits NFSR and a 128-bits LFSR. It relies also on two 64-bits accumulator and shift registers for MAC computation.

After 384 rounds of warm-up, Grain128-AEAD produces two streams of bits: the encryption keystream ks and the MAC keystream ms. They are extracted from the main keystream using bit parity i.e., $ks_i = y_{384+2i}$ and $ms_i = y_{384+2i+1}$ (where y_{383} denotes the last output bit from the warmup phase of the cipher).

6.3 MAC Computation

After Grain128a-AEAD warm-up for 384 rounds, the accumulator and the shift registers are initialized with the first 128-bits of keystream. Then, a message authentication code (MAC) can be computed for the plaintext bitstream m of size $M = |m|$. The MAC computation consists in updating the accumulator and the shift registers as follows:

$$a_k^{i+1} = a_k^i + m_i \cdot r_k^i, \quad 0 \leq i \leq M, \quad 0 \leq k \leq 63$$
$$r_k^{i+1} = r_{k+1}^i, \quad r_{63}^{i+1} = ms_i, \quad 0 \leq k \leq 62$$

where a_i^t and r_i^t denote the accumulator and shift register bits, respectively and ms the aforementioned MAC keystream. The MAC corresponds to the obtained accumulator at the end of this iteration.

Note that this MAC computation can be generalised to any stream-cipher, such as Trivium or Kreyvium, to extend it to support an authenticated encryption mode.

[3] Linear/Non-linear feeadback shift register.

6.4 MAC Usage in the Context of Transciphering

Now that we have a method to perform an authenticated encryption, the question of how to exploit the MAC after it has been homomorphically computed arises, i.e., how to fully achieve authenticated encryption within transciphering. So, when an authenticated encrypted message is received by the cloud it gets a message XORed by a portion of keystream and the associated MAC. Note that both of these are in the clear domain with respect to the FHE encryption layer. The cloud then homomorphically computes both an (homomorphic) encryption of the portion of keystream as well as (homomorphic) encryptions of the bits which *will* be used for (homomorphically) recomputing the MAC (from the message bits) as in Sect. 6.3. When appropriate, all these homomorphic ciphertexts can be precomputed offline. The cloud can then use the homomorphic keystream to retrieve (homomorphic) encryptions of the message bits which can then be used to homomorphically compute the associated MAC. The received MAC (a cleartext with respect to the FHE layer) can then be compared to the (homomorphically) computed one (a ciphertext with respect to the FHE layer) to lead to an (homomorphically) encrypted bit[4] equal to either 1 (if the MAC match) or 0 (otherwise). At this point, the cloud is by construction not able to know whether or not the two MAC match. Now assume that post transciphering the cloud need to (homomorphically) compute $f(m)$ or rather $[f(m)]$ (from $[m]$ which is the homomorphic encryption of the message obtained after transciphering). Then, if μ denotes the received MAC and $[\mu']$ the (homomorphically) computed one, we may want the cloud compute an encrypted result $[r]$ such that

$$[r] = \left[\begin{cases} f(m) \text{ if } \mu = \mu' \\ \text{NIL} \quad \text{otherwise} \end{cases} \right], \tag{1}$$

where NIL is out of f range. As such, post (FHE) decryption, the client can make the difference between valid outputs and invalid ones (due to integrity errors on the transmission channel). Note that, in order to comply with FHE requirements, Eq. (1), eventually has to be implemented by means of conditional assignments which can be performed by means of algebraic operations[5]. That way, of course, the cloud cannot distinguish between the case where it computed (an encryption of) NIL or (that of) a valid result.

7 Experimental Results

7.1 The Cingulata Homomorphic Compiler

Cingulata, formerly known as Armadillo [12], is a toolchain and run-time environment (RTE) for implementing applications running over homomorphic

[4] Of course, comparison cannot be done per se in the encrypted domain, so if the μ_i's denotes the bits of the received MAC and the $[\mu'_i]$'s denote the FHE encryptions of the bits of the (homomorphically) computed one, the cloud has to compute $\prod_i (1 \oplus \mu_i \oplus [\mu'_i]) = [b]$.

[5] I.e., putting a into x when c is true, b otherwise, can be written has $x := ca \oplus (1 \oplus c)b$.

encryption. Cingulata provides high-level abstractions and tools to facilitate the implementation and the execution of privacy-preserving applications.

Cingulata relies on instrumented C++ types to denote private variables, e.g., `CiInt` for integers and `CiBit` for Booleans. Integer variables are dynamically sized and are internally represented as arrays of `CiBit` objects. The Cingulata environment monitors/tracks each bit independently. Integer operations are performed using Boolean circuits, which are automatically generated by the toolchain. For example a full-adder circuit is employed to perform an integer addition. The Boolean circuit generation is configurable and two generators are available: focused on minimal circuit size or on small multiplicative-depth. More generally, it is possible to implement additional circuit generators or to combine them.

A `CiBit` object can be in either plain or encrypted state. Plain-plain and plain-encrypted bit operations are optimized out, in this way constant folding and propagation is automatically performed at the bit-level. Bit operations between encrypted values are performed by a "bit execution" object implementing the `IBitExec` interface. This object can either be a HE library wrapper, simply a bit-tracker object or even a plaint bit execution used for algorithm debugging purposes. When a HE library wrapper is used the Cingulata environment directly executes the application using the underlying HE library.

Another option is to use the bit-tracker in order to build a circuit representation of the application. The later allows to use circuit optimization modules in order to further optimize the Boolean circuit representation. The hardware synthesis toolchain ABC[6] is used to minimize circuit size. It is an open-source environment providing implementations of state-of-the-art circuit optimization algorithms. These algorithms are mainly designed for minimizing circuit area or latency but, currently, none of them is designed for multiplicative depth minimization. In order to fill this gap, several heuristics for minimizing the multiplicative depth are available in Cingulata, refer to [5,11] for more details.

The optimized Boolean circuit is then executed using Cingulata's parallel run-time environment. The RTE is generic, meaning that it uses a HE library wrapper, i.e. a "bit execution" object as defined earlier, in order to execute the gates of the circuit. The scheduler of the run-time allows to fully take advantage of many-core processors. Besides, a set of utility applications are provided for parameter generation (given a target security level), key generation, encryption and decryption. These applications are also generic, in the same vein as the parallel RTE.

7.2 Implementation Details

First, we did a simple implementation of Trivium, Kreyvium, Grain128a and Grain128-AEAD by following their original specifications using a binary representation [1,9,15,18]. We developed these algorithms with Cingulata as it provides all the necessary tools for implementing applications with a binary

[6] http://people.eecs.berkeley.edu/alanmi/abc/.

plaintext space, efficiently. Indeed, Cingulata specifies all the boolean operators needed for updating the bits of the internal state of each stream-cipher.

Second, we followed [6] and implemented each stream-cipher using longer words (8 or 32-bits long). Indeed, our selected stream-ciphers admit fairly efficient compact 8 and 32-bits software implementations which are suitable for more constrained platforms. Let us consider the example of Trivium, since it takes 64 (bit-level) cycles for the re-injection of t_3 to have an effect on t_1 [15], up to 64 cycles can be performed in parallel leaving the possibility for byte-oriented, 32-bit-word-oriented and 64-bits-word-oriented implementations requiring only 36 bytes of memory for the internal state (slightly more for 64-bits implementations as 288 is not a multiple of 64). The same analysis applies to Kreyvium, Grain128a and Grain128-AEAD.

Finally, we did a separate implementation for the MAC computation algorithm described in Sect. 6.3. Indeed, this MAC computation is adaptable to any stream-cipher once we are able to separate its keystream bits using their parity.

Dividing the keystream into two seperate keystreams was straightforward in Cingulata for the original implementation which outputs 1-bit of keystream at a time (i.e., at every keystream generation round). Indeed, we have just to take odd bits as the MAC computation keystream (ms in Sect. 6.3).

However, this task of keystream division using bits parity will become more challenging when we consider the compact 8-bits or 32-bits implementations without using Cingulata (which in fine works on a per-bits basis and, as such, gives an access to individual encrypted bits). When a true n-bits implementation is done, if our considered stream-cipher outputs 8-bits at a time, we will need to separate these 8-bits into 4-bits for encryption and 4 others for MAC computation. To do so, the most appropriate approach is to use a static look up table, with 256 entries, which outputs two 4-bits values using the initial (encrypted) 8-bits value as the selector.

7.3 Performance Results

As presented in Table 1, we see that the warm-up phase in each of the studied stream-ciphers remains time-consuming when performed homomorphically over TFHE (between 4 to 6 min). Trivium, Kreyvium and Grain128a remain somewhat inside the same range. This is due to the fact that Trivium and Kreyvium have a lightweight internal state update[7] but perform a large number of iterations (1152). On the other hand, Grain128a has a heavier update function[8] but only performs 256 iterations for warming-up. Lastly, for Grain128-AEAD the warm-up time significantly increases, because it does 128 additional iterations (384 in total) to initialize the accumulator and the shift register for MAC computation.

[7] A small number of additions and multiplications (only three) per round.

[8] LFSR/NFSR update, non linear function application and register shifting (register shifting being almost free).

However, following the operating mode defined in Sect. 4, when using TFHE, the warm-up phase can be performed (almost) once and for all and offline. As such, warm-up durations of a few minutes are easliy amortized and have no impact on latency. Hence, one can make sure, within a given protocol, that when some payload data is transmitted, the internal state of the stream-cipher, on the server side, has already been (homomorphically) warmed-up and is ready to produce the keystream bits required to transcipher that data. In fact, as a further optimization, the server may even start producing and internally storing homomorphically encrypted keystream bits as soon as the warm-up is done and prior to the reception of the symmetrically encrypted message. This results in an almost latency-free decompression process since only the remaining (homomorphic) XORs need to be done upon message arrival. Of course, the server still has to pay the computational cost of homomorphically computing the (FHE encrypted) keystream bits but that cost does not necessarily translate into additional latency.

Things are a little more complicated when authenticated encryption is implemented since, as discussed in Sect. 6.4, the MAC (homomorphic) computation has to be finalized when the message is received. Still, that finalization can be performed in parallel to the homomorphic evaluation of $f(\mathsf{FHE.Enc}(m))$ (the payload of the system). Upon completion of both, the two results can then combined following (1) for generating the final result.

With respect to our multi-bit implementations, recall that the Cingulata compiler automatically (and transparently to the programmer) turns them into (optimized) single-bit ones. As such, if Cingulata does its job properly we should expect that the comfort obtained by programming over words rather than bits does not translate into a degradation of the performances. Nicely, this is what we observed as the difference in execution times between single-bit and multi-bit implementations are not significant. Even if this test is slightly far-fetched for the considered algorithms (since they have natural single-bit expressions). This illustrates that Cingulata provides a higher level programming interface without significant losses in terms of performances (approximately +2% of computation time). See the two rightmost columns in Tables 1 and 3. Note additionally that the multi-bits implementation are more easily amenable to (automatic) parallelization.

We ran *single core* performance tests on an Intel(R) Xeon(R) CPU E3-1240 v5 @ 3.50 GHz and 8 GB RAM. However, contrary to the CTR mode of operation suggested in [9] (Sect. 4) we are now considering a more sequential more of operation. In the case where high parallelization is required to achieve low latency objectives we may thus need to interleave several keystreams (generated from several independant internal states). In that case, those keystreams can be computed in parallel (of course to the detriment of memory usage since several internal states would then have to be simultaneously stored).

We compare our results to the ones presented in [9]. In that paper, they used the BGV and BFV cryptosystems provided in HELib and Cingulata (respectively) with Trivium and Kreyvium, as well as LowMC which we do not consider in this paper (see Sect. 8). The parameters of the underlying FHE scheme

(levelled) were firstly selected to fit the exact multiplicative depth of the symmetric cryptosystem, which is not representative of a real-world scenario, since, obviously, the goal is to perform further computations on the decompressed ciphertexts. They then selected parameters for an additional multiplicative depth of 6 (i.e. for a total depth of 6 plus the multiplicative depth of either Trivium or Kreyvium). The single-slot and single-core throughputs (the number of keystream bits generated per minute) they obtained were between 3.15 and 3.66 bits/min[9]. Meanwhile, we obtain a throughput of around 120 bits/min, without any constraints on post-decompression computations, nor on the keystream size to be produced, and thus the size of the message to be decompressed. We use the default parameters of TFHE which provide a security level of 128-bits, and we ran the tests on a low end PC. In addition, [9] intensively used *batching* which is an optimization, not available in TFHE, consisting in packing several cleartexts in a single ciphertexts with FHE operations applying in a SIMD fashion. While in our work no major optimization were added than those automatically done by Cingulata. Also we would like to emphasize that Grain-128a and Grain128-AEAD would not be runable with a levelled FHE, at least not without a large increased in FHE parameters (in order to cope with the increase in multiplicative depth) which would dramatically impact performances. With that respect, this paper works changes the status for the homomorphic execution of Grain128a and Grain-AEAD from "practically undoable" to "only 3 times slower than Trivium/Kreyvium" (over TFHE).

Table 1. Warm-up time in minutes

	1-bit	8-bits	32-bits
Trivium	4.23	4.64	4.84
Kreyvium	4.98	5.52	5.83
Grain128a	4.12	4.15	4.21
Grain128-AEAD	6.55	6.68	6.71

8 Other Algorithms

In this work, we chose to investigate only stream-ciphers based on conservative designs and which are either standardized or in the process of being so. However, since the first proposal of transciphering as a technique for FHE ciphertext compression [7], many symmetric (block and stream) ciphers have been designed

[9] To be fair, these throughputs could be increased to around 2000 bits/min with 600 slots of batching, however with FHE security parameters which are now outdated (due to recent advances in LWE-based systems cryptanalysis); so as an order of magnitude we should expect all the throughputs given in [9] to be divided by around 5. Still, as already stated, batching-based improvements in throughput are not applicable when using TFHE.

Table 2. 64-bits stream generation time in minutes

	1-bit	8-bits	32-bits
Trivium	0.29	0.30	0.32
Kreyvium	0.33	0.36	0.39
Grain128a	1.09	1.05	1.06
Grain128-AEAD	1.13	1.12	1.14

Table 3. Homomorphic MAC computation time in minute of a 32-bits message

1-bit	8-bits	32-bits
0.65	0.66	0.68

in order explicitly fit FHE operations cost model, and as such, achieve improved FHE performances. These "FHE friendly" algorithms consider the requirement of leading to circuits with very few multiplications and a low multiplicative depth. FLIP [23], LowMC [2], FiLIP [20], and FASTA [14] are examples of new algorithms designed following this philosophy. Even though their efficiency in the FHE context is effective by construction, some of these forward-looking designs still have not yet stand the test of time in terms of security, at least compared to the algorithms which we considered in this paper (Table 2).

9 Conclusion

In this work we demonstrate the improvement brought by the TFHE cryptosystems and its practical bootstrapping capabilities to the issue of transciphering. In particular, on top on demonstrating much better homomorphic execution performances on stream ciphers already suitable for transciphering, this paper also demonstrates that additional algorithms and functionalities are within reach of practical homomorphic execution performances. This is the case, for the well-respected Grain algorithms family including their authenticated-encryption flavor which can further be adapted to pretty much any other stream cipher to enable them with message integrity checking.

Yet, even if we implemented and tested multi-bit versions of the algorithms, the Cingulata compiler has automatically turned them back to bit-wise implementation (without a significant performance loss which shows that the compiler does its job properly). As a result, this paper is only the first part of the story since we have thus (implicitly) limited ourselves to bit-wise implementations over TFHE in gate-bootstrapping mode (i.e. with $\mathbb{Z}_2$ as message space and a bootstrapping applied after each XOR and AND gates involved). The second part of the story which will be the scope of a subsequent paper will be to use the programmable bootstrapping feature of TFHE to design higher-level homomorphic operators allowing to run transciphering directly over larger message spaces (e.g., $\mathbb{Z}_{2^4}$ or $\mathbb{Z}_{2^8}$) with further efficiency gains.

References

1. Ågren, M., Hell, M., Johansson, T., Meier, W.: Grain-128a: a new version of Grain-128 with optional authentication. IJWMC **5**, 48–59 (2011)
2. Albrecht, M., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for MPC and FHE. Cryptology ePrint Archive Report 2016/687 (2016). https://eprint.iacr.org/2016/687
3. Albrecht, M.R., et al.: Feistel structures for MPC, and more. In: Sako, K., Schneider, S., Ryan, P.Y.A. (eds.) ESORICS 2019. LNCS, vol. 11736, pp. 151–171. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-29962-0_8
4. Albrecht, M.R., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for MPC and FHE. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9056, pp. 430–454. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46800-5_17
5. Aubry, P., Carpov, S., Sirdey, R.: Faster homomorphic encryption is not enough: improved heuristic for multiplicative depth minimization of Boolean circuits. In: Jarecki, S. (ed.) CT-RSA 2020. LNCS, vol. 12006, pp. 345–363. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-40186-3_15
6. Boudguiga, A., Letailleur, J., Sirdey, R., Klaudel, W.: Enhancing CAN security by means of lightweight stream-ciphers and protocols. In: Romanovsky, A., Troubitsyna, E., Gashi, I., Schoitsch, E., Bitsch, F. (eds.) SAFECOMP 2019. LNCS, vol. 11699, pp. 235–250. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-26250-1_19
7. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference, ITCS 2012, pp. 309–325. Association for Computing Machinery, New York (2012). https://doi.org/10.1145/2090236.2090262
8. Canteaut, A., et al.: Stream ciphers: a practical solution for efficient homomorphic-ciphertext compression. In: FSE, pp. 313–333 (2016)
9. Canteaut, A., et al.: Stream ciphers: a practical solution for efficient homomorphic-ciphertext compression. J. Cryptol. **31**(3), 885–916 (2018). https://doi.org/10.1007/s00145-017-9273-9
10. Canteaut, A., et al.: Stream ciphers: a practical solution for efficient homomorphic-ciphertext compression. J. Cryptol. **31**(3), 885–916 (2018). https://doi.org/10.1007/s00145-017-9273-9. https://hal.inria.fr/hal-01650012
11. Carpov, S., Aubry, P., Sirdey, R.: A multi-start heuristic for multiplicative depth minimization of Boolean circuits. In: Brankovic, L., Ryan, J., Smyth, W.F. (eds.) IWOCA 2017. LNCS, vol. 10765, pp. 275–286. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78825-8_23
12. Carpov, S., Dubrulle, P., Sirdey, R.: Armadillo: a compilation chain for privacy preserving applications. In: Bao, F., Miller, S., Chow, S.S.M., Yao, D. (eds.) Proceedings of the 3rd International Workshop on Security in Cloud Computing, SCC@ASIACCS 2015, Singapore, Republic of Singapore, 14 April 2015, pp. 13–19. ACM (2015). https://doi.org/10.1145/2732516.2732520
13. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: fast fully homomorphic encryption over the torus. Cryptology ePrint Archive Report 2018/421 (2018). https://eprint.iacr.org/2018/421
14. Cid, C., Indrøy, J.P., Raddum, H.: FASTA - a stream cipher for fast FHE evaluation. Cryptology ePrint Archive Report 2021/1205 (2021). https://ia.cr/2021/1205

15. De Cannière, C., Preneel, B.: Trivium. In: Robshaw, M., Billet, O. (eds.) New Stream Cipher Designs. LNCS, vol. 4986, pp. 244–266. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-68351-3_18
16. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing, STOC 2009, pp. 169–178. ACM, New York (2009). https://doi.org/10.1145/1536414.1536440
17. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: conceptually-simpler, asymptotically-faster, attribute-based. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013. LNCS, vol. 8042, pp. 75–92. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40041-4_5
18. Hell, M., Johansson, T., Maximov, A., Meier, W., Yoshida, H.: Grain-128AEADv2: strengthening the initialization against key reconstruction. Cryptology ePrint Archive Report 2021/751 (2021). https://ia.cr/2021/751
19. Hoffmann, C., Méaux, P., Ricosset, T.: Transciphering, using FiLIP and TFHE for an efficient delegation of computation. In: Bhargavan, K., Oswald, E., Prabhakaran, M. (eds.) INDOCRYPT 2020. LNCS, vol. 12578, pp. 39–61. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-65277-7_3
20. Hoffmann, C., Méaux, P., Ricosset, T.: Transciphering, using FiLIP and TFHE for an efficient delegation of computation. Cryptology ePrint Archive Report 2020/1373 (2020). https://eprint.iacr.org/2020/1373
21. Lepoint, T., Naehrig, M.: A comparison of the homomorphic encryption schemes FV and YASHE. In: Pointcheval, D., Vergnaud, D. (eds.) AFRICACRYPT 2014. LNCS, vol. 8469, pp. 318–335. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-06734-6_20
22. Méaux, P., Carlet, C., Journault, A., Standaert, F.-X.: Improved filter permutators for efficient FHE: better instances and implementations. In: Hao, F., Ruj, S., Sen Gupta, S. (eds.) INDOCRYPT 2019. LNCS, vol. 11898, pp. 68–91. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-35423-7_4
23. Méaux, P., Journault, A., Standaert, F.-X., Carlet, C.: Towards stream ciphers for efficient FHE with low-noise ciphertexts. In: Fischlin, M., Coron, J.-S. (eds.) EUROCRYPT 2016. LNCS, vol. 9665, pp. 311–343. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49890-3_13
24. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. J. ACM **56**(6) (2009). https://doi.org/10.1145/1568318.1568324